AI 과정 정리 발표 · 10/12 – 10/16

AI 학습 정리 1–5일차

LLM 기초 → RAG → Fine-tuning → AI 에이전트 → MCP

2조: 맹지현, 이승윤, 왕선규, 정현우, 김동준
발표자: 이승윤

전체 로드맵

4일 동안의 흐름: 이해 → 주입 → 수정 → 위임

1일차 · 10/12

LLM 기초

LLM은 무엇이고
어떻게 호출하나?

토큰 · temperature · stateless

→

2일차 · 10/13

RAG

LLM이 모르는 지식을
어떻게 주나?

청킹 · 임베딩 · 벡터 DB

→

3일차 · 10/14

Fine-tuning

모델 자체를 어떻게
내 용도에 맞추나?

LoRA · QLoRA · 하이퍼파라미터

→

4일차 · 10/15

AI 에이전트

LLM에게 어디까지
맡기고 어떻게
통제하나?

패턴 · 도구 · 트레이스

각 일차는 **앞 일차에서 드러난 LLM의 한계**를 해결하는 방향으로 이어집니다.

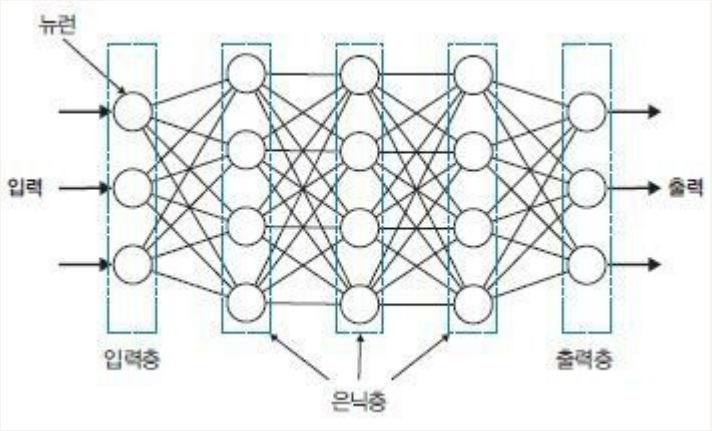
LLM은 다음 토큰을 예측하는 딥러닝 모델

인공지능(AI) · 판단·학습·추론 기술 전체

머신러닝 · 데이터에서 규칙을 스스로 찾음

딥러닝 · 다층 인공 신경망을 사용

LLM · 대량의 글을 학습한 거대 모델



학습 원리

순전파(예측)

↓

손실 계산

↓

역전파(틀린 원인 추적)

↓

경사하강법(가중치 수정)

다음 토큰 예측

입력 "하늘은 왜" 다음에 올 토큰 후보

파란	**62%**
높은	15%
흐린	8%

토큰을 하나씩 골라 이어 붙여 문장을 완성합니다.

temperature 0
항상 1위 선택 → 일관

temperature 1
확률대로 선택 → 창의

1일차 · 호출 구조

LLM 호출은 4단계로 흐르고 매번 백지에서 시작

STEP 1

메시지 구성

system 개발자의 지시
user 사용자의 말
assistant 모델의 답

STEP 2

API 호출

base_url·api_key만 바꿔 OpenAI·Gemini·Ollama 같은 형식으로 호출. 키는 .env에 보관

STEP 3

모델 생성

다음 토큰 예측 반복
temperature 무작위성
max_tokens 생성 상한

STEP 4

응답 확인

content 답변 본문
finish_reason
stop 정상 · length 잘림

Stateless
이전 호출을 기억하지 못함

기억 구현 (STEP 4 → 1)
지난 대화를 messages에 누적해 다시 전송

Context Window
한도를 넘으면 오래된 대화부터 잘림

토큰 비용 · 한국어는 토큰을 더 쓰고, 비용은 출력에서 더 큼 → "답은 세 문장 이내로"

1일차 → 2·3일차 연결

LLM의 한계와 이를 푸는 세 가지 방법

학습 이후 정보를 모름
예: 어제 공개된 CVE

비공개 정보를 모름
예: 우리 회사 보안 규정

환각 (Hallucination)
모르는 것도 그럴듯하게 지어냄

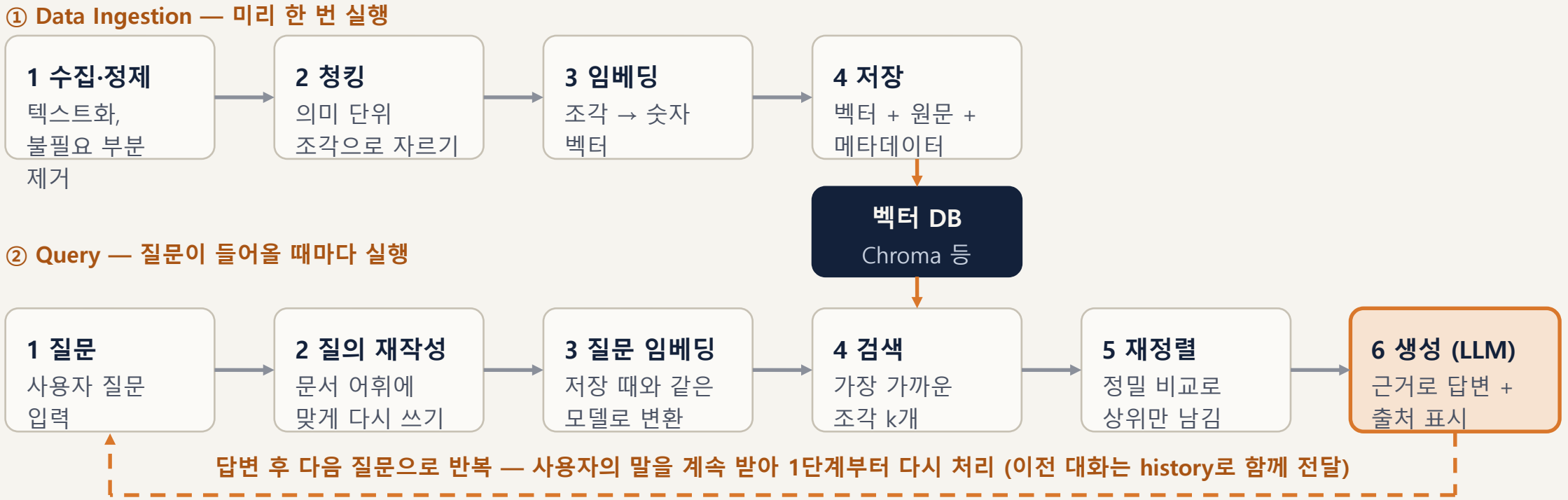
기억 없음 · 분량 한도
Stateless, Context Window

① 프롬프트에 직접 넣기
가장 간단하지만 문서가 많으면 다 못 넣음

② RAG → 2일차
필요한 부분만 검색해서 넣음 · 출처 제시 가능

③ Fine-tuning → 3일차
내 데이터로 모델 자체를 추가 학습

RAG는 두 개의 파이프라인으로 움직인다



벡터 DB는 근거를 **찾아 줄 뿐**, 정리해서 답하는 것은 LLM — 품질은 대부분 **검색**에서 결정된다

2일차 · 단계별 핵심

RAG 품질을 좌우하는 네 단계

01 청킹 — 한 조각에 한 이야기

고정 크기 · Recursive(기본값) · Semantic · 문서 구조 · LLM 기반.
너무 작으면 문맥이 잘리고, 너무 크면 검색이 흐려짐 → overlap으로 보완

02 임베딩 · 벡터 DB

의미가 비슷하면 가까운 좌표(코사인 유사도) → "암호"로 "비밀번호"도 찾음. 대량 벡터는 ANN(HNSW)으로 거의 정확하고 빠르게.
메타데이터로 필터·출처 표시 가능

03 검색 · 재정렬

가까운 k개(2~5)를 무조건 가져옴 → 재정렬로 상위만 LLM에 전달.
CVE 번호처럼 정확 일치가 필요하면 하이브리드(BM25 + 의미)

04 생성 · 평가

"자료만 근거로", "없으면 없다고", "[1], [2] 출처 표시", temperature 0.
정답 라벨 질문 세트(채점표)로 숫자 평가

Advanced RAG · 질의 재작성, 하이브리드 검색, 재정렬, 점수 하한선으로 단계마다 개선

Agentic RAG · 에이전트가 검색 여부·횟수·도구를 스스로 결정 → 4일차로 연결

3일차 · Fine-tuning 진행 단계

Fine-tuning은 9단계, 그중 4–7을 반복한다



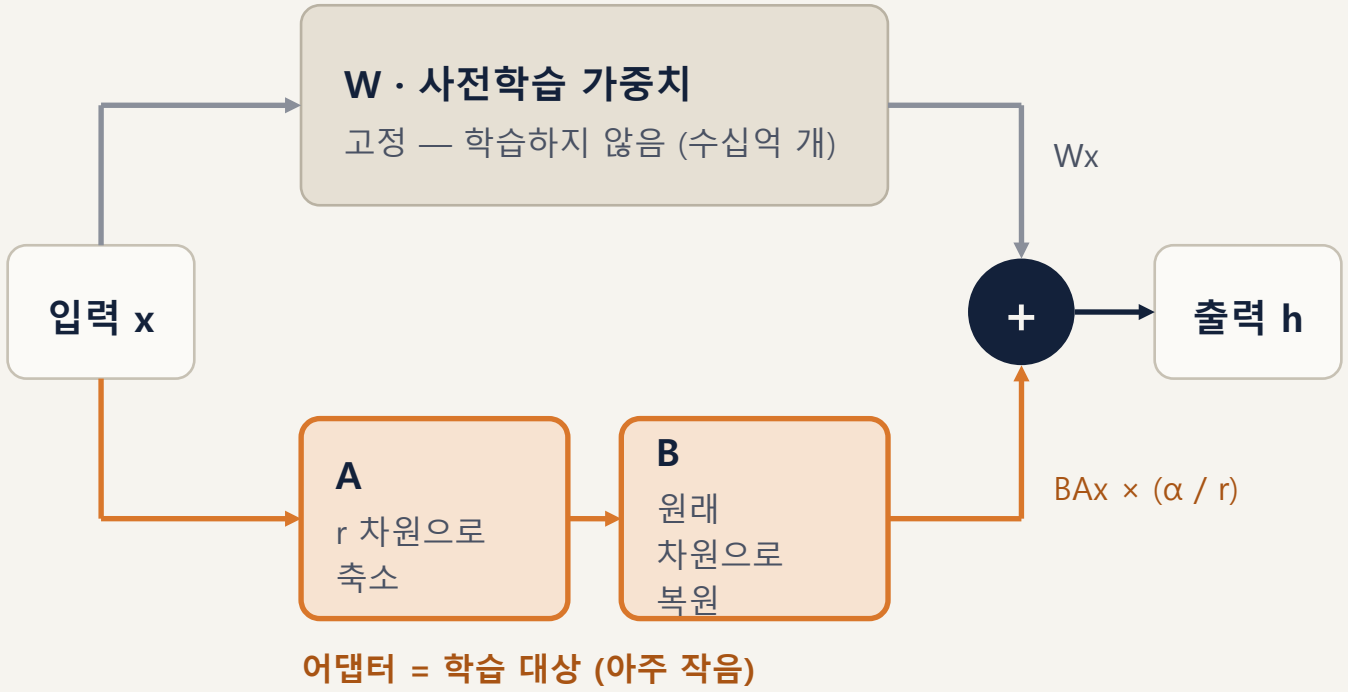
파라미터 · 데이터로 학습되는 모델 내부 값 (가중치·편향)

하이퍼파라미터 · 사람이 학습 전에 정하는 값 (학습률, r, alpha, 에폭)

오픈 웨이트 · Qwen·Llama처럼 가중치만 공개된 모델을 받아 미세조정

기준	RAG	Fine-tuning
바꾸는 것	프롬프트에 넣는 자료	모델의 가중치
강한 분야	최신·사내 지식, 출처 (자주 바뀌는 지식)	말투·형식·특정 작업 (고정된 행동·형식)
정보 갱신	DB만 갱신	재학습 필요
비용	학습 불필요	GPU·시간 필요

LoRA: 원래 가중치는 고정, 작은 어댑터만 학습



주요 하이퍼파라미터

이름	의미
r	어댑터 폭, 주로 8~16
alpha	어댑터 출력 배율(영향력)
learning rate	가중치 수정 정도

QLoRA = LoRA + 4bit 양자화

가중치를 16bit → 4bit(정규분포 기반)로 줄여
메모리 절감 → Colab같은 제한된 GPU에서도 학습

장점 · 메모리 절감 · 작은 결과물 · 작업별 어댑터 교체 · 추론 지연 없음 · 전체 미세조정에 가까운 성능

Anthropic
에이전트는 일을 위임하는 것, 핵심은 통제

워크플로 (Workflow)

LLM과 도구가 **미리 정한 코드 경로**대로 조율됨 → 예측·통제 쉬움

에이전트 (Agent)

LLM이 **작업 절차와 도구 사용을 스스로 결정** → 유연하지만 추적 어려움

누가 결정하나? — 영업 메일 실습의 세 가지 방식

① 코드가 결정

순서 고정 · 디버깅 쉬움 · 가장 안전

② 에이전트 + 도구

매니저 LLM이 도구를 골라 실행 · 지시문만 바꾸면 됨

③ 핸드오프

결정까지 다른 에이전트에 넘김 · 예측 가장 어려움

통제·예측 쉬움

유연성 ↑ · 통제 어려움 → 트레이스 필수

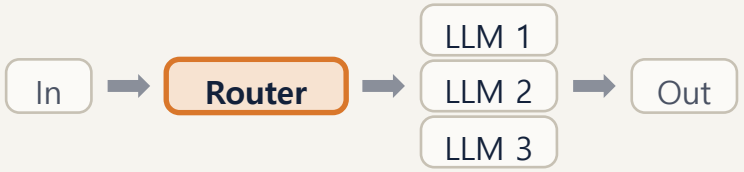
패턴마다 LLM이 결정하는 범위가 다르다

프롬프트 체이닝



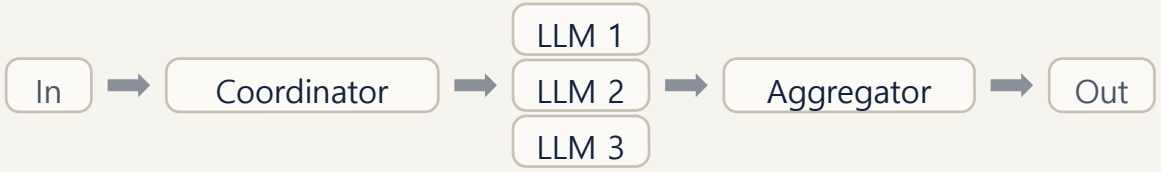
단계를 순서대로 잇고 중간에 검증

라우팅



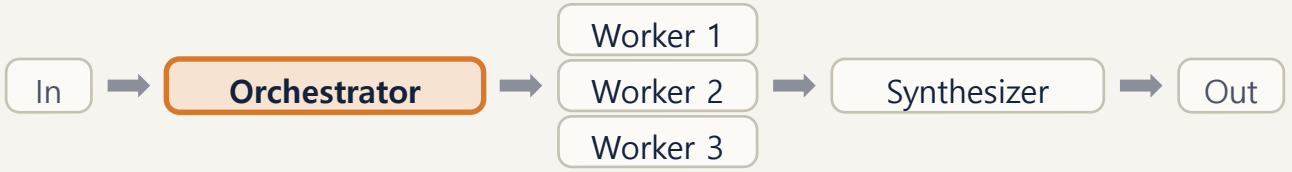
입력을 보고 경로 하나를 선택

병렬화



동시에 나눠 처리하고 결과를 모음

오케스트레이터-워커



작업을 동적으로 쪼개 맡기고 종합

평가자-최적화



통과할 때까지 생성-평가 반복

▢ = LLM이 다음 흐름을 결정하는 지점 · 나머지 연결은 코드가 고정 · **단순한 설계가 이긴다**

에이전트는 도구로 행동하고, 트레이스로 확인한다



트레이스 — 위 모든 단계의 판단과 도구 호출을 기록 → 문제 원인 추적 · 비용(크레딧) 확인

하네스 · 도구만으로 부족하면 파일 시스템 · 플래닝(가장 중요) · 서브에이전트 · 스킬을 갖춘 실행 환경을 붙인다

5일차 · MCP 기초

MCP는 AI에 도구를 꽂는 USB 포트

AI 쪽 · 클라이언트

주피터 노트북 · VS Code
Claude 앱

에이전트는 지시만 하고, 검색·파일 저장 같은 실제 일은 도구가 한다

MCP 연결 규격

로컬 stdio

원격 URL
(Streamable HTTP)

도구 쪽 · MCP 서버

내 PC에서 uvx · npx · Docker로 실행

fetch · Playwright · filesystem · Context7 · Tavily

도구가 도는 곳 ≠ 자료가 있는 곳

fetch는 내 PC에서 돌지만 읽는 것은 인터넷 페이지

AI가 잘 읽는 형식

JSON · Markdown · XML — 도구 결과도 이 형식으로 주고받는다

만들지 말고 가져다 쓴다

AI 개발은 속도전. 교수님 추천 서버: GitHub, Chrome DevTools

5일차 · MCP 실습

도구를 꽂으면 순서는 AI가 알아서 정한다

01 mcp_basic

떡 조리법 검색 → ricecake.md 저장

순서를 알려 주지 않아도 AI가 도구를 골라 실행

02 Context7

꽂기 전엔 "모른다"

꽂은 뒤엔 최신 문서를 찾아서 답함

03 Docker 샌드박스

mcp/fetch · mcp/filesystem을 상자 안에서 실행

--mount로 허락한 폴더만 연결

04 메모리 MCP

새 대화에서도 이름·좋아하는 떡을 기억

기억도 모델이 아니라 도구가 한다

05 Agentic RAG

Tavily 검색 + Qdrant 벡터DB

"검색해서 저장해 줘" 한마디로 검색 → 저장

06 시장 데이터 MCP

API 권한 부족으로 조회 실패

유료 데이터는 API 키·요금제부터 확인

결론 · 코드는 거의 같고, 꽂는 서버만 바뀐다

5일차 · fetch vs Playwright

JS로 그리는 페이지는 진짜 브라우저가 필요하다

fetch 웹 페이지 → Markdown 변환

85 자

같은 취약점 페이지에서 가져온 글자 수

"불러오는 중..."만 수집 · CVE 번호 없음

가볍고 빠르지만 JS 실행 전의 화면만 읽음

Playwright 진짜 브라우저로 페이지를 띄움

222 자

같은 취약점 페이지에서 가져온 글자 수

CVE-2021-44228 · CVSS 10.0 · 조치 방법까지 수집

무겁지만 JS가 그린 실제 내용을 읽음

사용 순서

fetch로 먼저 시도 → 내용이 비었나 확인 → **Playwright로 전환**

가벼운 도구부터 시작한다

5일차 · 내 도구 만들기

도구 3개로 취약점 보고서를 자동 작성

assets.md
자산 목록 읽기

→

list_cves
등록된 CVE 확인

→

lookup_cve
CVE 상세 조회

→

버전 비교
영향 여부 판단

→

report.md
보고서 저장

주황 = 직접 만든 cve_server.py의 도구 · 나머지는 filesystem MCP로 읽고 쓰기

결과 — 두 자산 모두 "영향 있음"

Log4j 2.14.1 → CVE-2021-44228

Cisco AnyConnect 4.9.0 → CVE-2024-20337

샌드박스 밖 파일 읽기 시도 → **서버가 거부**

"기억으로 답하지 말고 도구로 조회해" 한 줄이 정확도를 지킨다

이미 만든 코드도 도구가 된다

`@server.tool()` 함수 → MCP 도구
독스트링 = AI가 읽는 설명서

`@function_tool` 보안 RAG → 에이전트 도구
전체를 실행하는 맨 위 함수에 붙인다

`Agent(mcp_servers=, tools=)`
남의 도구와 내 도구를 함께 사용

정리 · 1–5일차

어디까지 맡기고, 어떻게 통제할 것인가

1일차	LLM은 다음 토큰 예측기, 기억이 없다
2일차	RAG는 근거를 찾아 넣는다, 품질은 검색이 좌우
3일차	Fine-tuning은 가중치를 고친다, LoRA로 가볍게
4일차	에이전트는 위임이다, 통제는 코드와 트레이스로
5일차	MCP로 도구를 꽂아 쓴다, 위험한 건 샌드박스에서

MCP 사용 5원칙 · 5일차

01 **꽂아 쓴다** 코드는 같고 서버만 바뀐다

02 **로컬이라 안전하다는 착각** 외부 연동은 샌드박스에서

03 **검증된 MCP만** API 키·비밀번호는 함부로 넘기지 않기

04 **도구 목록부터 확인** 유료 API·GPU 비용도 함께 생각하기

05 **직접 만들어 봐야 안다** 남의 도구를 쓰더라도 원리는 손으로 익힌다

에이전트 개발 5원칙 · 4일차

1. 실행 권한은 우리에게 있다
모델은 "부르겠다"고 말할 뿐, 실행은 코드가 한다.

2. 코드로 될 일은 코드로
가장 흔한 실수는 코드로 충분한 일을 LLM에게 맡기는 것.

3. 안전장치는 코드에 둔다
되돌릴 수 없는 동작일수록 더욱.

4. 말이 아니라 기록으로 확인
최종 응답이 아니라 도구 호출 기록(트레이스)을 본다.

5. 독스트링이 곧 명세
모델은 함수 본문을 보지 않는다.

그리고 무엇보다, 기록(문서화)이 가장 중요하다

감사합니다 · 질문 받겠습니다